**EXPERIMENT-02 :**

**For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples**

**CODE**

```
# Candidate-Elimination Algorithm

# Dataset + Algorithm + Matplotlib Output


import pandas as pd

import matplotlib.pyplot as plt


# 1. Create the dataset

data = {

    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],

    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],

    'Humidity': ['Normal', 'High', 'High', 'High'],

    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],

    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],

    'Forecast': ['Same', 'Same', 'Change', 'Change'],

    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']

}


df = pd.DataFrame(data)


print("Training Dataset:")

print(df)
```

```
# Save dataset as CSV
df.to_csv("EnjoySport.csv", index=False)

# 2. Separate features and target
X = df.iloc[:, :-1].values.tolist()
Y = df.iloc[:, -1].tolist()

n = len(X[0])

# 3. Initialize Specific and General boundaries
S = ['Ø'] * n
G = [['?'] * n]

# 4. Candidate-Elimination Algorithm
for i, example in enumerate(X):

    if Y[i] == 'Yes':

        # Remove G hypotheses inconsistent with positive example
        G = [
            g for g in G
            if all(g[j] == '?' or g[j] == example[j]
                   for j in range(n))
        ]
```

```
    # Generalize S to cover positive example
    for j in range(n):
        if S[j] == 'Ø':
            S[j] = example[j]
        elif S[j] != example[j]:
            S[j] = '?'

else:

    # Specialize G for negative example
    new_G = []

    for g in G:
        if all(g[j] == '?' or g[j] == example[j]
               for j in range(n)):

            for j in range(n):
                if g[j] == '?':
                    values = df.iloc[:, j].unique()

                    for value in values:
                        if value != example[j]:
                            h = g.copy()
                            h[j] = value
                            new_G.append(h)
        else:
```

```
                new_G.append(g)

        G = new_G

    print(f"\nAfter Example {i + 1}:")
    print("S =", S)
    print("G =", G)

# 5. Display final boundaries
print("\nFinal Specific Boundary (S):")
print(S)

print("\nFinal General Boundary (G):")
for g in G:
    print(g)

# 6. Matplotlib visualization
hypotheses = [S] + G
row_labels = ['Specific Boundary S'] + [
    f'General Boundary G{i+1}' for i in range(len(G))
]

fig, ax = plt.subplots(figsize=(12, 4))
ax.axis('off')
ax.set_title(
    'Candidate-Elimination Algorithm - Final Hypothesis Boundaries',
```

```
    fontsize=14,
    pad=20
)

table = ax.table(
    cellText=hypotheses,
    rowLabels=row_labels,
    colLabels=df.columns[:-1],
    cellLoc='center',
    loc='center'
)
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.1, 1.8)
plt.tight_layout()
plt.show()
# 7. Download CSV dataset
from google.colab import files
files.download("EnjoySport.csv")
```

**OUTPUT:**

[ ?, ?, ?, ?, Cool, ? ]

Candidate-Elimination Algorithm - Final Hypothesis Boundaries

| | Sky | AirTemp | Humidity | Wind | Water | Forecast |
|---|---|---|---|---|---|---|
| Specific Boundary S | Sunny | Warm | ? | Strong | ? | ? |
| General Boundary G1 | Sunny | ? | ? | ? | ? | ? |
| General Boundary G2 | ? | Warm | ? | ? | ? | ? |
| General Boundary G3 | ? | ? | ? | ? | Cool | ? |